AERO60492 – Autonomous Mobile Robots

Coursework 1 – Serial message decoding

Section 1: aims, timeline, submission requirements, marking scheme.

Aims

The aim of this task is to familiarise yourself with the principals of data encoding standards for transmission as a serial message. Reformatting information into a sequential stream of 'bytes' (encoding) followed by the splitting of the stream back into its components (decoding) is the basic principal that vast amounts of our civilisations technology is built upon. It underpins the internet with protocols such as TCP/IP, computing with protocols such as USB, and robotics with protocols such as ROS[1] topics or MAVLink[2] messages.

The task will be to interpret a serial message definition table and create a program to split an example set of data back into its constituent values. In other words, using a protocol definition, decode a serial data stream into human readable values.

Timeline

- Week 1 – coursework is set with accompanying lecture
- Week 2 – work sheet on serial protocols
- Week 3 – submission

Submission

The required submission a Python3 script which can open your encoded binary data file and decode its contents using the specified serial message protocol into the human readable values following the example given. Commenting of the code should be used to explain your logic applied to the decoding process and use of any functions. A few lines of comments should be used to answer the additional questions at the top of your script file.

Additionally, a text file containing the decoded values in a .csv format should be submitted.

The filenames should use your student number as the file name. e.g.

`'11223344.py'`

`'11223344.csv'`

Marking scheme

This coursework has a 10% contribution to the unit overall. Marks out of 20 will be awarded according to:

- Successful message decoding into human readable output [10 marks]
- Explanation of the method used (in comments) [4 marks]
- Answers to additional questions (in comments) [3 marks]
- Good coding practices [3 marks]

---

[1] https://wiki.ros.org/Topics
[2] https://mavlink.io/en/

Section 2: pre-requisites and background info.

Pre-requisites

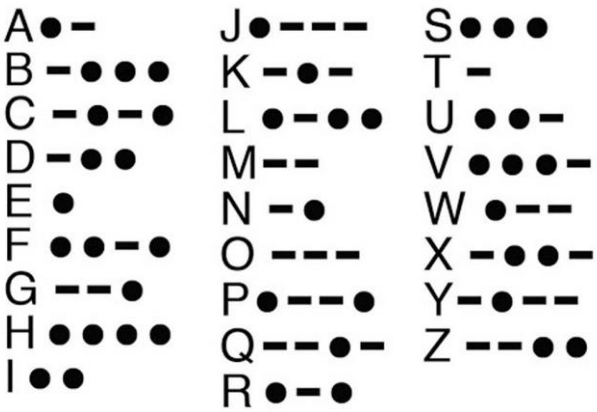
It is expected that you already have a good understanding of the Python programming language. This work does not necessarily require the use of advanced programming methods or libraries. It can use basic file operations (i.e. open/read/close), flow control (i.e. if/else/while statements), some data type conversions (i.e. decimal/hex/byte/character), and basic arithmetic value manipulations to achieve the required outcome. For a refresher on these topics please visit:

- Interacting with binary files: https://www.geeksforgeeks.org/reading-binary-files-in-python/
- **Converting a multiple bytes into integer values: https://www.geeksforgeeks.org/how-to-convert-bytes-to-int-in-python/ and the https://docs.python.org/3/library/stdtypes.html for the 'int.from_bytes' method.**
- Displaying output messages in console: https://realpython.com/python-print/
- Flow control: https://docs.python.org/3/tutorial/controlflow.html

There are many different more advanced options for how to perform the task below with the Python language which is left to you to explore as needed.

Background

Encoding and serialising data can find its origins in protocols such as smoke signals or Morse code. For the former starting in 1500 BCE a simple pattern of one smoke puff for "I am here", two puffs for "all is ok" and three puffs for "there is something wrong" could allow people to communicate over vast distances benefiting from what is effectively speed of light data transmission. Due to the need to clearly separate puffs by hand and the weather dependent nature of atmospheric visibility, the bandwidth was extremely limited with just one piece of information transmitted every 30 seconds or so. Jumping forwards to the 19th century saw the invention of telegraphy with a popular method of encoding text known as Morse code. With this method a more sophisticated protocol was defined as a series of short (dot) and long (dash) pulses which could be sent over telephone lines or radio frequencies using a hand operated 'Morse key'.



The Morse code character definitions[3]

[3] . −..− − .−. .− ..... −.− ..− −.. −−− ... .−. ..... −.−− − −− ..− ..... − . .−.. .−.. ..... −− . ..... − .... −− .− −− . .. −.−. ..... .−− −−− .−. −.. ....−.. . .−−. .... .− −. −

You might recognise the ubiquitous sequence of pulses • • • – – – • • • (dot dot dot dash dash dash dot dot dot) as 'SOS' signal for distress, or the very similar • • • – – • • • used for old Nokia text message (aka 'SMS') alerts. The invention of Morse code now allowed complete paragraphs of text to be sent over even greater distances with skilled operators able sending (encoding) and receiving (decoding) 20-30 words per minute.

Fast forward to the computing era, serial message protocols are now used to communicate everything as a series of bits and bytes. Early computing used parallel type communications which allowed the simultaneous transmission of one byte as eight parallel bits. Since one byte can be used to represent 256 different unique combinations in binary form, parallel communication worked well when most data were text or numbers (e.g. a text printer), and hence the ASCII[4] standard was created to ensure both the encoding and decoding devices interpreted the bytes symmetrically. Later these bits started to be transmitted in series along one wire with standards such as UART (universal asynchronous receiver / transmitter) and USB (universal serial bus) becoming widespread.

The modern USBC standard is orders of magnitude higher bandwidth than the older standards (e.g. 80 Gbps for USB4 V2 vs 8 Mbps for ECP parallel)

Sending large amounts of numerical data between computing devices has a drawback. Values which are already stored as binary data must first be converted into text characters, transmitted, interpreted, and converted back into bytes. As an example, lets send the three RGB (red, green, blue) values that can represent one pixel colour. Each value is already one byte (8 bits) with a value between 0-255. Lets use the University of Manchester purple with values [109, 000, 157]. In binary form this is just three bytes. To send as text, we must send at least nine characters sequentially, i.e. ['1' '0' '9' '0' '0' '0' '1' '5' '7'] and use triple the bandwidth and additional computing flops to perform the conversions at either end of the communication. It would be much more beneficial if the data remined in binary form.

Have you ever tried opening an image file in notepad? It might have looked something like this….

[4] https://www.youtube.com/watch?v=ahnfLZKwnTg

But it is actually a picture of an apple…

The reason it appears as nonsensical text is because it a binary encoded format. This means the simple direct conversion of bytes to text using the ASCII standards is not enough. You might be familiar with the names of many image encoding formats such as JPEG, PNG, GIF, TIFF to name a few. For each of these standards there is a defined convention of how the RGB values have been written directly to the file as binary values.

To send this binary image to another location requires splitting the data into bytes, transmitting the bytes in order over a serial data connection, then reconstructing the image file byte-by-byte at the receiver. This would create an identical copy of the binary data, but an interpreter is still required to view the image. The interpreter takes the bytes and decodes the values into a more useful form. Modern internet connections and video streaming can now perform these tasks seamlessly at very high data rates over very long distances. For example, a zero compression 4K 60 fps video stream is approximately 12 Gbps with all 12 billion bits being processed to screen in 1/60th of a second. The bandwidth of the average home in the UK is only 70 Mbps, hence additional image compression algorithm is used to reduce the bandwidth requirements.

When transmitting a ‘serialised’ data packet as a stream of sequential bytes, both the encoder (sender) and decoder (receiver) devices need to share a common understanding of which order the bytes are transmitted. Watchout for the ‘endianess’ of the bytes when performing your decoding.

Section 3: task briefing, questions, and examples

Task

Using the following message protocol, create a Python3 script to open a binary data file and decode the data into a series of values. Each file will have multiple sequential data frame entries and each entry will have its bytes arranged according to:

	Byte	Value	Meaning	Interpretation
Header	1	~	STX1	Two sequential ~ characters indicate the start of a new data frame
	2	~	STX2	
	3	0-255	SYS ID	An integer value indicating which device sent this data 0-255
	4	0-255	DEST ID	An integer value indicating the device this data was destined for 0-255
	5	0-255	COMP ID	An integer value indicating which sub-component sent this data 0-255
	6	0-255	SEQ	A value which should increment by one each data frame. Used to allow the detection of missing frames.
	7	0-255	TYPE	An ID value indicating what type of data is in the payload.
Payload	8	P	PTX	A value indicating the end of the message header and start of the payload.
	9	0-255	RPM MSB	Most and least significant bytes for RPM measurement as unsigned 16-bit unsigned integer. Unit: rev per minute
	10	0-255	RPM LSB	
	11	0-255	VLT MSB	Most and least significant bytes for voltage measurement as unsigned 16-bit unsigned integer. Unit: millivolt
	12	0-255	VLT LSB	
	13	0-255	CRT LSB	Least and most significant bytes for current measurement as signed 16-bit integer. Unit: milliamp
	14	0-255	CRT MSB	
	15	0-255	MOS TMP	Temperature of mosfets. Use lookup table to interpret value.
	16	0-255	CAP TMP	Temperature of capacitors. Use lookup table to interpret value.
Timing	17	T	TTX	A value indicating the end of the message payload and indicating the start of the timing information.
	18	0-255	TME B0	A timestamp expressed as a Unix time code as a 64 bit unsigned integer. B0 is most significant bit, B7 is least significant bit. Unit: microseconds
	19	0-255	TME B1	
	20	0-255	TME B2	
	21	0-255	TME B3	
	22	0-255	TME B4	
	23	0-255	TME B5	
	24	0-255	TME B6	
	25	0-255	TME B7	
Checksum	26	0-255	CHKSUM	Checksum value used to indicate if a data frame has become corrupted.

The temperature lookup table is:

Byte value	Temperature C						
0xA0	30	0xA1	30.1	0xA2	30.2	0xA3	30.3
0xA4	30.4	0xA5	30.5	0xA6	30.6	0xA7	30.7
0xA8	30.8	0xA9	30.9	0xAA	31	0xAB	31.1
0xAC	31.2	0xAD	31.3	0xAE	31.4	0xAF	31.5
0xB0	31.6	0xB1	31.7	0xB2	31.8	0xB3	31.9
0xB4	32	0xB5	32.1	0xB6	32.2	0xB7	32.3
0xB8	32.4	0xB9	32.5	0xBA	32.6	0xBB	32.7
0xBC	32.8	0xBD	32.9	0xBE	33	0xBF	33.1
0xC0	33.2	0xC1	33.3	0xC2	33.4	0xC3	33.5
0xC4	33.6	0xC5	33.7	0xC6	33.8	0xC7	33.9
0xC8	34	0xC9	34.1	0xCA	34.2	0xCB	34.3
0xCC	34.4	0xCD	34.5	0xCE	34.6	0xCF	34.7
0xD0	34.8	0xD1	34.9	0xD2	35	0xD3	35.1
0xD4	35.2	0xD5	35.3	0xD6	35.4	0xD7	35.5
0xD8	35.6	0xD9	35.7	0xDA	35.8	0xDB	35.9
0xDC	36	0xDD	36.1	0xDE	36.2	0xDF	36.3

The input data binary file can be downloaded from Blackboard. **Use the binary file which matches your name in the "FileAllocationList.xlsx" spreadsheet. These files are individual to each of you.**

Header

Bytes 1-2: The first values of each data frame are always the same. Note that the data file might not start exactly at the boundary of a complete data frame, hence the code must be able to locate the start of one frame and decode the following characters. Any partial data frames need to be discarded as corrupt or incomplete.

Bytes 3-5: These are simple one-byte values (i.e. 0-255) which act to identify where the data originated and where is should be sent. This this is similar to how data packets on the internet will often include the IP addresses which allows for processes and hardware to check data is being routed to the correct destination.

Byte 6: A sequence number which should increment by 1 each data frame. If two sequential frames have a non-sequential number in this byte, then it indicates an entire frame might have been missed and a receiving process can take appropriate action. Note that when the value reaches 255 (the maximum for one byte) should reset to 0 on the next data frame.

Byte 7: A value indicating the type of data which will follow. This value will often require a documentation lookup. For example in the MAVLink[5] protocol a message ID of 16 indicates a waypoint location data frame, and a value of 24 is a GPS data frame.

[5] https://mavlink.io/en/messages/common.html

Payload

Byte 8: Another character to indicate the start of the payload bytes has been reached.

Bytes 9-10, 11-12, 13-14: Pairs of bytes which should be combined into signed or unsigned integers. Since numerical values often exceed 0-255, we must use 16-bit values which allow a greater range of 0-65535 for unsigned and -32768-32767 for signed integers.

Bytes 15-16: Two values which require a further processing step using a conversion table. The table provided should be used to find the correct physical value.

Time:

Byte 17: A constant character to indicate the start of the timing bytes has been reached.

Bytes 18-25: These are 8 bytes which should be combined into a single 64-bit integer. It is a timestamp expressed in microseconds since a universal date in the past known as the Unix epoch[6]. Since the number of microseconds will be in the quadrillions range (e.g. 1738589377864000 for when I wrote this), hence require a 64-bit number.

Checksum

Byte 26: The final value(s) in many data protocols is known as a checksum. It is a value which is computed based upon all the bits in the message and allows for messages which have been corrupted during transmission to be detected. For example, when using radio communications in very 'noisy' environments.

There are many options for checksums which can allow for different levels of data assurance and reliability. A basic checksum algorithm has been implemented using the following steps:

1) Add up all the values in the message (including the two STX (%) characters and excluding the checksum value) expressed as one-byte decimal integers.
2) Take the modulo[7] of this sum against a value of 256.
3) Subtract the result from 255.
4) The result is the checksum value.

This process was used to create the data frame checksum values and should be implemented in your decoder script as a method to identify corrupt data frames. i.e. if your calculated frame checksum value doesn't match the value in byte 26, then something might be wrong with the values within it, hence it is a corrupt frame.

[6] https://en.wikipedia.org/wiki/Unix_time
[7] https://en.wikipedia.org/wiki/Modulo

Additional questions

Using your decoder script, answer these additional questions. The answers need to be provided as comment lines at the start of your Python files. Including them in the decoded file or as other text in the submission will not be accepted.

1) How many complete data frames are in your file?
2) How many messages might be corrupt?
3) What is the calendar date of these messages?

Skeleton Code

The aim of this task is to implement a serial message decoding function, hence a 'skeleton' code is provided to handle the input file manipulations and read the binary data one byte at a time (i.e. as would happen with other forms of binary data communication which is transmitted one byte at a time).

Your main task is to fill in the section which interprets each incoming byte, but the entire file can and must be edited as needed to add variables for other things (i.e. counting). To complete the task, you will need to also add code to create an output file and keep track of any values needed to answer the questions.

DO NOT change this code to read the entire file into memory at the start. The idea is that this is a stream of bytes is mimicking a communication stream. So just as they would arrive byte by byte on a serial connection, the skeleton script reads them byte by byte.

```
#Start of skeleton code
#Q1 Answer: ??
#Q2 Answer: ??
#Q3 Answer: ??

#Open the binary input file
input_file = open("<your binary file name>", 'rb')

#Read the first byte and loop as long as
#there is always another byte available
byte = input_file.read(1)
while byte:
      print("Byte value is (hexidecimal): " + str(byte))
      print("Byte value is (decimal): " + str(int.from_bytes(byte)))

      #
      #
      # add decoding method here
      #
      #

      #Get the next byte from the file and repeat
      byte = input_file.read(1)


#Must be end of the file so close the file
print("End of file reached")
input_file.close()

#End of skeleton code
```

It is expected you'll need to edit more than just the central section (i.e. maybe some global variables or storage buffers as you see fit). How complex you make it depends on your skill and interest. The skeleton code is provided to ensure you could read a series of sequential bytes as a functioning starting point, and to demonstrate some basic byte manipulations. Python is a very flexible language; hence this decoding problem can be solved many ways. Use of functions, classes, libraries, arrays, lists, additional variables, if/else/for, etc.... are all up to you to decide. But remember, there are more marks for neat and functionally working code than fancy code.

Example Output lines

The output file will use the comma separated variable format whereby each item has a ',' between, and a new line per frame. Each line should be the decoded and interpreted binary values directly converted from the file. The checksum output should be the value contained at byte26 converted to decimal - not your own calculated checksum. If the value for temperature is outside the lookup table range, this might indicate that value has become corrupted – you can use the checksum for that frame to verify this. For any temperature values outside the lookup, please put '0.0' in the output file. If you have identified a corrupt frame, still output the potentially 'wrong' values as a line and still count this frame in the total number of complete frames.

```
#Start of example output .csv file
~~,219,247,7,27,25,P,2532,9136,-66,30.9,31.0,T,970479533016225,209,
~~,219,247,7,28,25,P,2653,7298,4293,34.7,31.9,T,970479534016225,178,
~~,219,247,7,29,25,P,2615,2582,2234,32.1,32.4,T,970479535016225,235,
~~,219,247,7,30,25,P,3470,6712,2098,35.3,35.7,T,970479536016225,19,
~~,219,247,7,31,25,P,2751,5500,-2649,32.2,31.5,T,970479537016225,251,
~~,219,247,7,32,25,P,2773,8466,-43,32.1,32.2,T,970479538016225,114,
<more lines.....>
#End of example output lines
```

Don't forget to upload both the decoded output file and your Python script file to Canvas, e.g.

'11223344.py'

'11223344.csv'